



愛芯 AX650N 模型轉換及移植教學 (本地編譯)

作者：SDP Alex、Christine

0. 前置準備

(1) 創建并進入 Conda 環境

```
conda create -n axmodel python=3.10 -y  
conda activate axmodel
```

(2) 安裝 PyTorch

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
```

(3) 安裝 Ultralytics 和 ONNX Runtime

```
pip install ultralytics onnxruntime numpy onnx-simplifier onnx
```

(4) 在開始前，請確保模型已經轉換成 .onnx，並經過 onnxsim 優化:

Yolo 模型可使用 ultralytics 指令列 (CLI) 進行快速轉換：

```
yolo export model=yolo11m.pt format=onnx opset=12 simplify=True imgsz=640
```

在轉換完成後，通常會用 onnxruntime 載入模型並進行驗證。這種「轉換+驗證」的自動化

流程無法用單一的 CLI 指令完成。所以建議用 Python 腳本轉換模型，可參考附件：

```
convert_model_yolo11m.py
```

如有需要，可對 onnxsim 優化進行 5 次隨機輸入檢查：

```
onnxsim yolo11m.onnx yolo11m_sim.onnx 5
```

會得到優化後的 yolo11m_sim.onnx

創建 model 資料夾，將轉為 onnx 的模型存入 model/目錄下。

(5) (如有需要) ONNX 模型切割

使用 Netron 工具 (<https://netron.app/>) 載入 ONNX 檔。點擊節點查看輸入/輸出名稱。

識別計算密集塊 (如 Conv 層群) 結束處的中間張量作為切割點。 (自行編寫並運行 Cut.py)

(6) 工具鏈映像檔：

愛芯官方提供、選擇適合版本的 Pulsar2 Docker 映像檔壓縮包。

<https://hf-mirror.com/AXERA-TECH/Pulsar2/tree/main> (如: ax_pulsar2_4.0.tar.gz)

(7) 校準數據集：

創建 dataset 資料夾，在 dataset/目錄下準備：

一個包含 50-100 張的數據集，並已打包成 .tar 格式 (dataset.tar)。

個人建議可以使用 val dataset 其中的 50-100 張。

(8) 準備 pulsar2-run-helper 資料夾

複製 pulsar2-run-helper 資料夾

(9) 測試圖片：

pulsar2-run-helper/sim_images/目錄下至少一張用於最終仿真驗證的清晰圖片 (00001.jpg)。

(10) 工作目錄結構

/SDK/	
└─ pulsar2/	← 專案根目錄
└─ ax_pulsar2_4.0.tar.gz	← 愛芯 Pulsar2 SDK v4.0 的工具鏈壓縮檔
└─ model/	
└─ yolov11m.onnx	← 已轉換好的 ONNX 格式模型檔案
└─ config/	
└─ yolov11m_build_config.json	← 模型轉換工具 (pulsar_converter) 所需的設定檔
└─ dataset/	
└─ dataset.tar	← 用於模型量化 (PTQ) 的校準資料集壓縮檔
└─ pulsar2-run-helper/	← PC 端仿真輔助工具資料夾
└─ model/	← 存放仿真時所需模型的資料夾
└─ pulsar2_run_helper/	← 官方提供的 PC 端仿真輔助工具資料夾
└─ list.txt	← (新增) 存放待處理圖片路徑的清單文件
└─ cli_classification.py	← 用於「分類模型」前後處理的命令列腳本
└─ cli_detection.py	← 用於「偵測模型」前後處理的命令列腳本
└─ sim_images/	
└─ 000001.jpg	← 單張測試圖片範例
└─ scripts/	← 存放各種腳本的資料夾
└─ convert_model_yolo11m.py	← 用於模型轉換的腳本

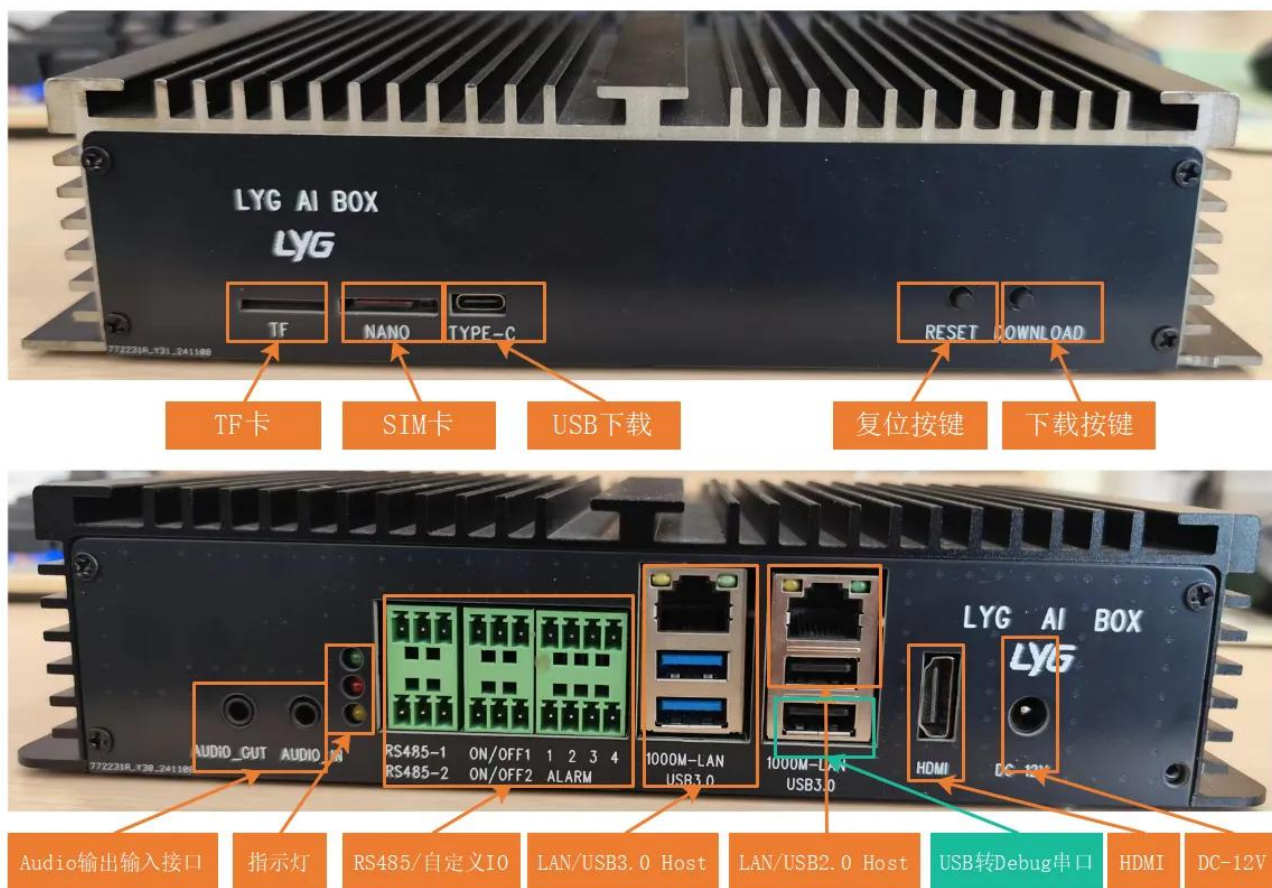
(11) 硬體清單

1. AX650 開發板: 包含主板、散熱片等。
2. 電源供應器
3. 網路線
4. RTSP 相機
5. HDMI 線
6. 滑鼠&鍵盤

(12) 軟體清單

1. 燒錄工具: AXDL_V1.24.13.1.7z
2. 工具驅動: Driver_V1.20.46.1.7z
3. 固件 :
AX650_emmc_ubuntu_rootfs_desktop_V3.6.2_20250603154858_20250807105950_DDR4+4G.axp
4. LYG 版編譯源碼及 SDK : lyg-samples.tag.gz

(13) 接口介绍



- 接口可参考 LYG AI BOX

1. 安裝 Pulsar2 工具鏈

(1) 安裝 Docker 開發環境

[Windows | Docker Docs](#)

[Linux | Docker Docs](#)

Docker 成功安裝後，輸入指令：

```
sudo docker -v  
Docker version 20.10.7, build f0df350
```

顯示以上內容，說明 Docker 已安裝成功。下面將介紹 Pulsar2 工具鏈 Image 的安裝啟動。

(2) 載入 Docker Image

輸入指令：

```
sudo docker load -i ax_pulsar2_4.0.tar.gz
```

導入 docker 鏡像文件。正確導入鏡像文件

完成後，輸入指令：

```
docker image ls
```

看到工具鏈鏡像成功載入後便可以基於此鏡像啟動容器

(3) 啟動工具鏈鏡像

執行

```
sudo docker run -it --net host --rm -v $PWD:/data pulsar2:4.0
```

2. 模型轉換

(1) 編寫 build 配置文件

在 docker 容器內，根据参数规格编写 json 格式的配置文件。

於當前目錄下創建 config 資料夾，並開放權限：

```
mkdir config  
chmod 777 ./config
```

在 config/目錄下創建：**yolo11m_sim_build_config.json**

(請以實際 model 命名，此處以 yolo11m_sim.onnx 為例)

JSON 設定檔完整版參考：**bulidjson.json** 檔案

這邊以 yolo11m_sim_build_config.json 為例：

```
{  
  "model_type": "ONNX",  
  "npu_mode": "NPU1",  
  "quant": {  
    "input_configs": [  
      {  
        "tensor_name": "images",  
        "calibration_dataset": "./dataset/dataset_yolo11.tar",  
        "calibration_size": 32,  
        "calibration_mean": [0, 0, 0],  
        "calibration_std": [255.0, 255.0, 255.0]  
      }  
    ],  
    "calibration_method": "MinMax",  
    "precision_analysis": true,  
    "precision_analysis_method": "EndToEnd"  
  },  
  "input_processors": [  
    {  
      "tensor_name": "images",
```

```

        "tensor_format": "BGR",
        "src_format": "BGR",
        "src_dtype": "U8",
        "src_layout": "NHWC"
    }
],
"output_processors": [
    {
        "tensor_name": "/model.23/Concat_output_0",
        "dst_perm": [0, 2, 3, 1]
    },
    {
        "tensor_name": "/model.23/Concat_1_output_0",
        "dst_perm": [0, 2, 3, 1]
    },
    {
        "tensor_name": "/model.23/Concat_2_output_0",
        "dst_perm": [0, 2, 3, 1]
    }
],
"compiler": {
    "check": 0
}
}

```

(2) 執行編譯

在容器內的 /data 目錄下，執行以下 build 指令。

```
pulsar2 build --target_hardware AX650 --input model/yolo11m_sim.onnx --output_dir output
--config config/yolo11m_sim_build_config.json
```

(3) 檢查結果

編譯成功後，會在 output/ 目錄下生成 compiled.axmodel 檔案。

3. 模型仿真驗證

(1) 準備仿真檔案

```
cp output/compiled.axmodel pulsar2-run-helper/models/yolo11m_sim.axmodel
```

進入仿真工具目錄

```
cd pulsar2-run-helper/
```

(2) 編寫 CLI 脚本

pulsar2-run-helper 中包含 cli_detection.py 和 cli_classification.py 兩個 CLI 脚本範例，需要根據模型應用自行編寫對應的 CLI 脚本（如：cli_pose.py）。

(3) 執行前處理和推理

前處理：生成 .bin 輸入檔 (letterbox_size 根據輸入圖片大小調整)

```
python3 cli_detection.py --pre_processing --image_path sim_images/00001.jpg --axmodel_path models/compiled.axmodel --intermediate_path sim_inputs/0
```

推理：生成 .bin 輸出檔

確認 list.txt 內容是 --intermediate_path sim_inputs/0 輸入的名稱（此處為 0）

```
pulsar2 run --model models/compiled.axmodel --input_dir sim_inputs --output_dir sim_outputs --list list.txt
```

(4) 執行驗證脚本

```
python3 cli_detection.py --post_processing --image_path sim_images/00001.jpg --axmodel_path models/compiled.axmodel --intermediate_path sim_outputs/0
```

仿真驗證成功後，輸入 **exit** 離開 docker 環境

4. 連接 AX650 並設定網路

(1) 硬體連接

1. 將電源接上 AX650 開發板
2. 用網路線將開發板的網路孔連接到您的路由器
3. 用 USB-A 對 USB-A 線連接 AX650 USB 轉 Debug 串口 (AX650 接口標示圖中綠色標示) 和電腦
4. HDMI 線接到螢幕
5. 滑鼠&鍵盤接到 USB3.0

(2) 燒錄固件步驟

1. 於 PC 端解壓縮 AXDL_V1.24.13.1.7z 和 Driver_V1.20.46.1.7z

2. 安裝燒錄工具驅動：

(1) Driver_V1.20.46.1 → DriversForWin10 → 執行 DriverSetup.exe

Drivers	2020/11/11 15:31	文件夹	
Utils	2020/11/11 15:31	文件夹	
☒ DriverSetup.exe	2020/11/11 11:30	应用程序	316 KB
DriverUninstall.exe	2020/11/11 11:29	应用程序	315 KB

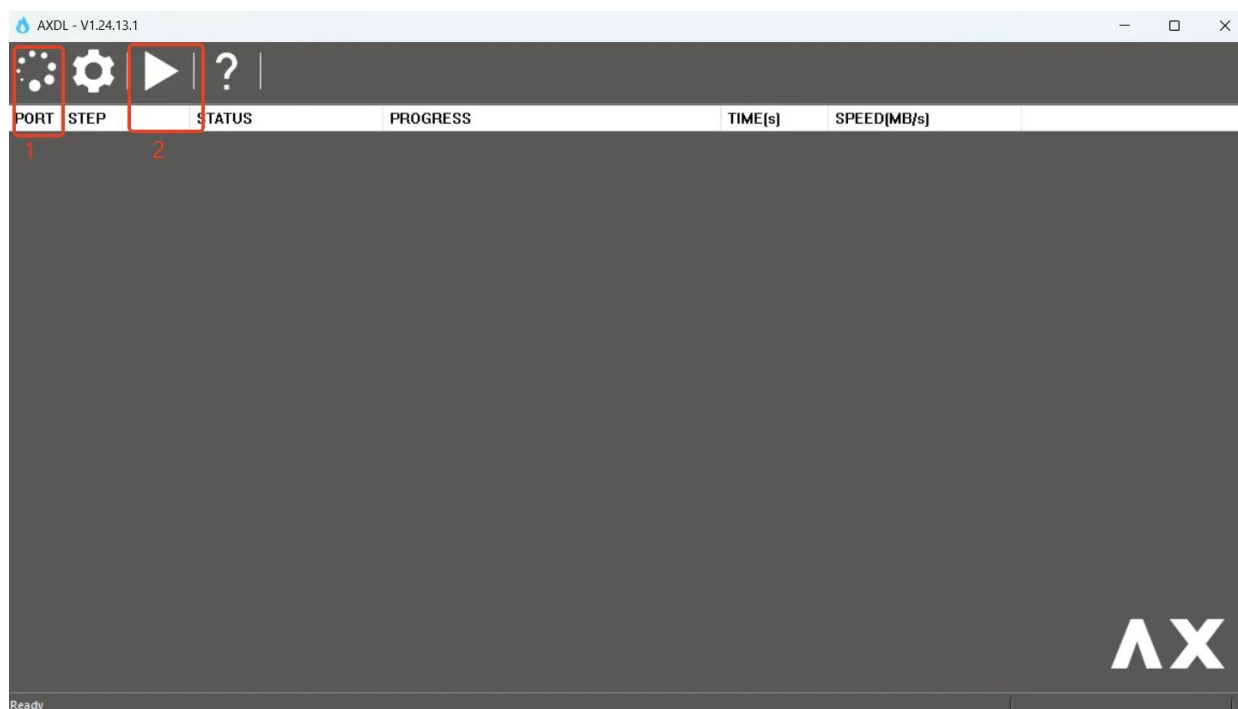
(2) Driver_V1.20.46.1 → DriversForWin10 → Utils → 執行 DPInst64.exe

config.reg	2020/11/11 11:41	注册表项	1 KB
dpinst.xml	2020/11/11 11:26	XML Document	1 KB
DPInst32.exe	2019/4/16 16:43	应用程序	901 KB
☒ DPInst64.exe	2019/4/16 16:43	应用程序	1,026 KB
DriverUninstall32.exe	2020/11/11 11:37	应用程序	23 KB
DriverUninstall64.exe	2020/11/11 11:37	应用程序	28 KB

3. 進入燒錄工具資料夾 AXDL_V1.24.13.1 → Bin · 運行 AXDL.exe

config	2024/3/25 16:25	文件夹	
ReadFlash	2023/7/27 12:33	文件夹	
7z.dll	2021/6/28 10:43	应用程序扩展	1,115 KB
7zDll.dll	2021/6/28 10:43	应用程序扩展	41 KB
☒ AXDL.exe	2024/3/25 16:24	应用程序	3,960 KB
AXDL.ini	2023/8/11 14:21	配置设置	4 KB

4. 開始燒錄



1: 選擇需要燒錄的固件:

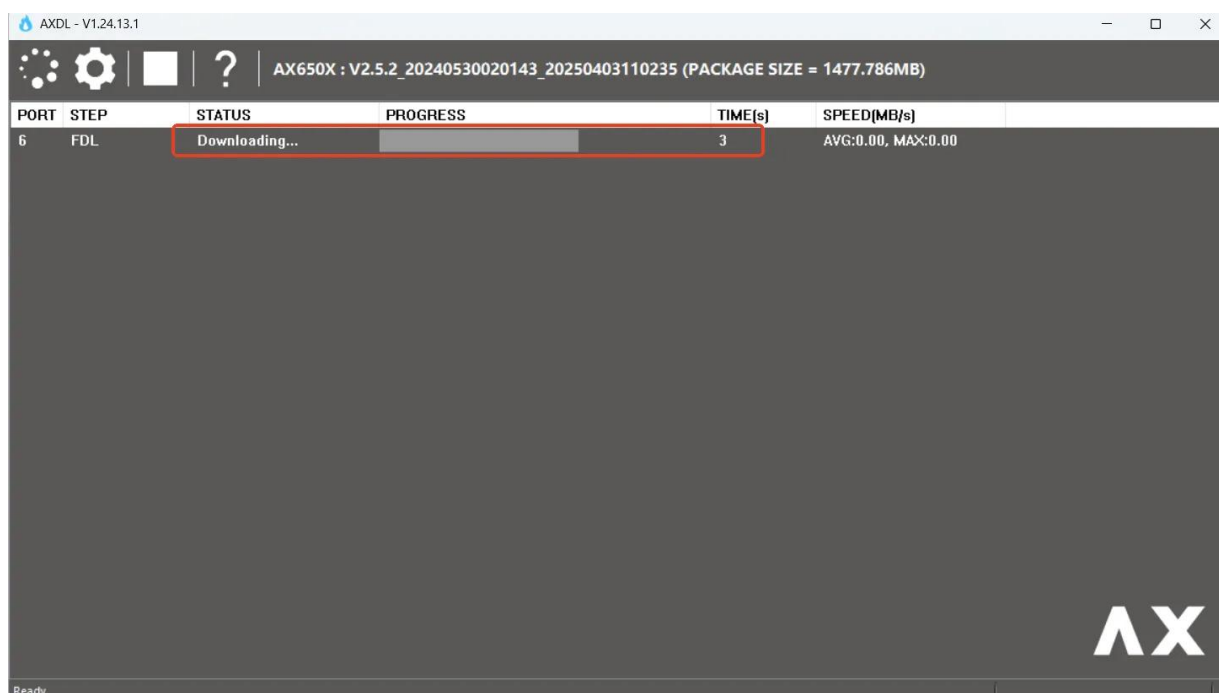
AX650_emmc_ubuntu_rootfs_desktop_V3.6.2_20250603154858_20250807105950_DDR4+4G.axp

2: 點擊開始燒錄

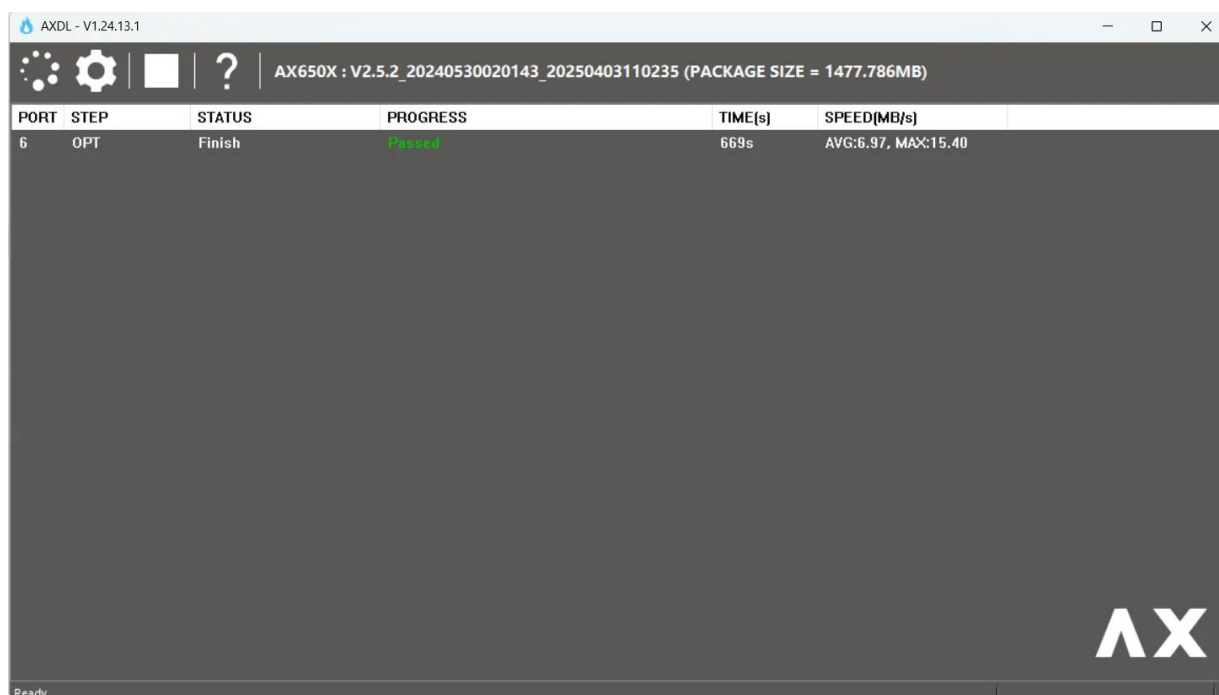
3: 同時按住 RESET 和 DOWNLOAD 按鍵，然後鬆開 RESET 按鍵，等待設備進入下載模式，AXDL 工具顯示正在下載後，再鬆開 DOWNLOAD 鍵。



4: 正在下载中...

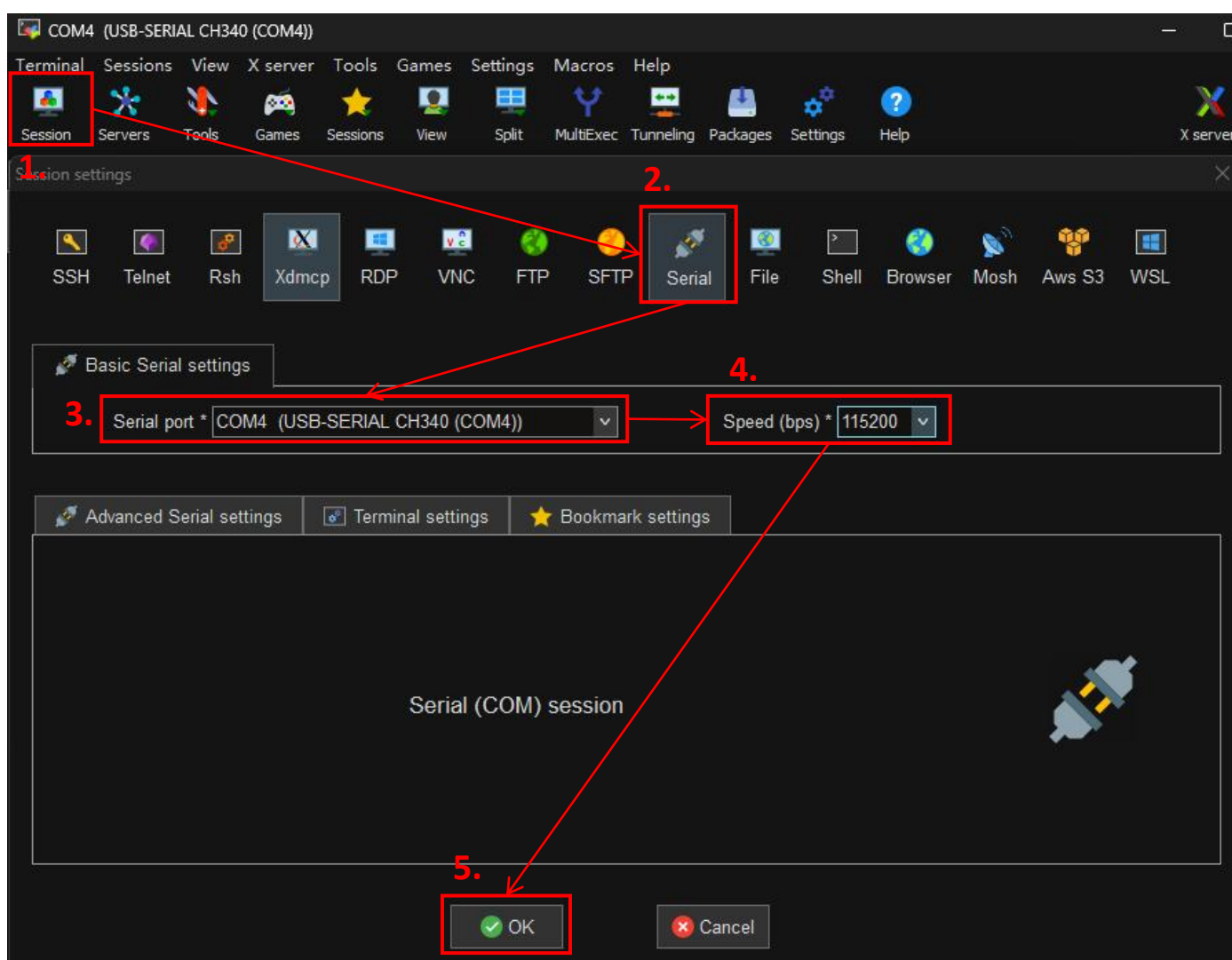


5: 下载完成



(3) 尋找並連接序列埠

1. 開啟 MobaXterm 中的 Session
2. 點選 Serial
3. 選擇 Serial Port · 通常只有一個選擇 (當前連接的 650 的 port)
4. 選擇 Speed 為 115200
5. OK



(4) 設定網路

確認 AX650N 的 IP：在 AX650N 的 Serial Port 序列埠終端機中，輸入以下指令：

```
ip addr show eth0
```

```
root@ax650:~/lyg-sample/build/install/ax650# ip addr show eth0
3: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether e6:6a:d7:37:24:fb brd ff:ff:ff:ff:ff:ff
    inet 172.19.30.227/24 brd 172.19.30.255 scope global eth0
        valid_lft forever preferred_lft forever
root@ax650:~/lyg-sample/build/install/ax650#
```

可以找到 AX650N 的 IP，此處為：172.19.30.227。確保 IP 前 3 組數字和相機 RTSP IP 一樣。

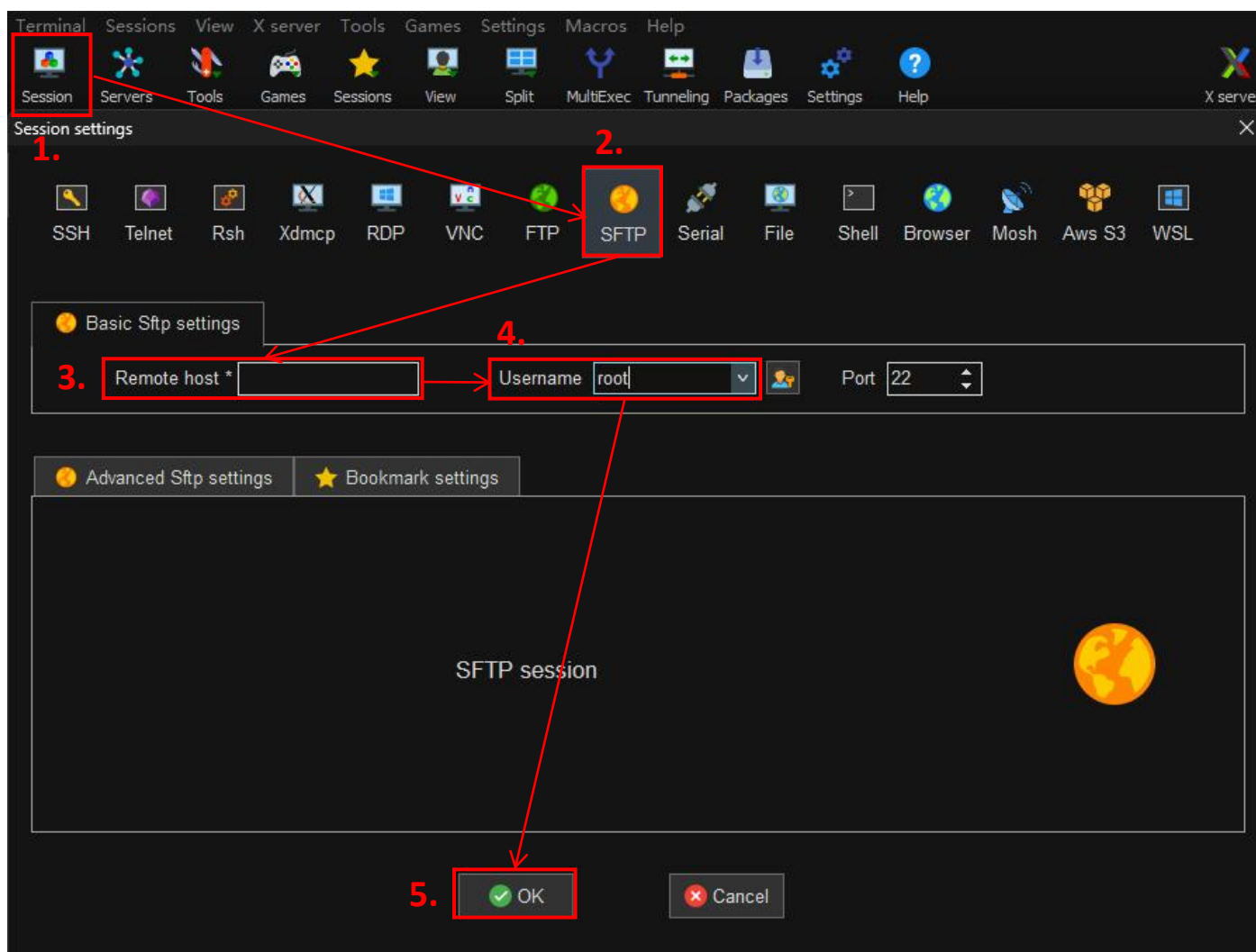
(5) 環境搭建

在 Serial Port 序列埠終端機中輸入：

```
apt update
apt install build-essential libopencv-dev cmake
```

(6) 開啟 SFTP 傳輸檔案

Session->SFTP → 在 Remote host 輸入 AX650N 的 IP 地址 → Username 輸入 root → OK



(7) 準備編譯源碼及 SDK 和需要編譯的文件

1. 在 SFTP 中把 lyg-samples.tag.gz 上傳到 AX650N 的 root/目錄下
2. 回到 Serial Port 序列埠終端機中，輸入以下指令解壓縮：

```
tar -xzf lyg-samples.tag.gz
```

3. 解壓縮後目錄為以下內容：

```
lyg-samples$ tree -L
.
├── AX650_SDK_V1.45.0_P39
├── benchmark
├── build
│   ├── CMakeFiles
│   ├── examples
│   └── install
│       └── ax650
├── cmake
├── docs #範例程式執行所需.jpg, .png 圖片文件
│   └── ax650
├── examples
│   ├── ax650 #需編譯之程式.cc 存放位置
│   ├── deprecated
│   ├── middleware
│   ├── lyg_falldetect.cc
│   ├── yolo11detect.cc
│   ├── lyg_ball_detect.cc
│   ├── lyg_face_detect.cc
│   ├── lyg_helmet_nv12.cc
│   ├── ...
│   └── CMakeLists.txt
├── base/
│   ├── common.hpp
│   ├── detection.hpp #通常需要根據不同模型做出修改
│   ├── pose.hpp
│   ├── score.hpp
│   ├── topk.hpp
│   ├── transform.hpp
│   └── yolo.hpp
├── utilities/
│   ├── args.hpp
│   ├── cmdline.hpp
│   ├── file.hpp
│   ├── plit.hpp
│   └── timer.hpp
```

4. 設定環境路徑，開啓 ./bashrc 文件

```
vim ./bashrc
```

5. 進入文件後在最後一行加入以下路徑

```
export CFLAGS="-I/root/lyg-sample/AX650_SDK_V1.45.0_P39/msp/out/include"  
export CXXFLAGS="-I/root/lyg-sample/AX650_SDK_V1.45.0_P39/msp/out/include"  
export LDFLAGS="-L/root/lyg-sample/AX650_SDK_V1.45.0_P39/msp/out/lib"
```

6. 參考範例修改.cc，並把要編譯的.cc 程式文件放到 lyg-sample/examples/ax650 中

7. 視所使用的模型情況，有機會需要修改 detection.hpp 等頭文件

8. 如要編譯新的.cc 程式碼，請在 lyg-samples/examples/ax650/CMakeLists.txt

加入以下代碼：

```
axera_example(project_name.cc)
```

(8) 編譯文件

```
cd lyg-sample/ build/  
cmake -DBSP_MSP_DIR=/root/Desktop/lyg-samples/examples/ax650 AXERA_TARGET_CHIP=ax650 ..  
make -j6  
make install
```

編譯完成後，生成的可執行示例存放在 root/lyg-sample/build/install/ax650 路徑下

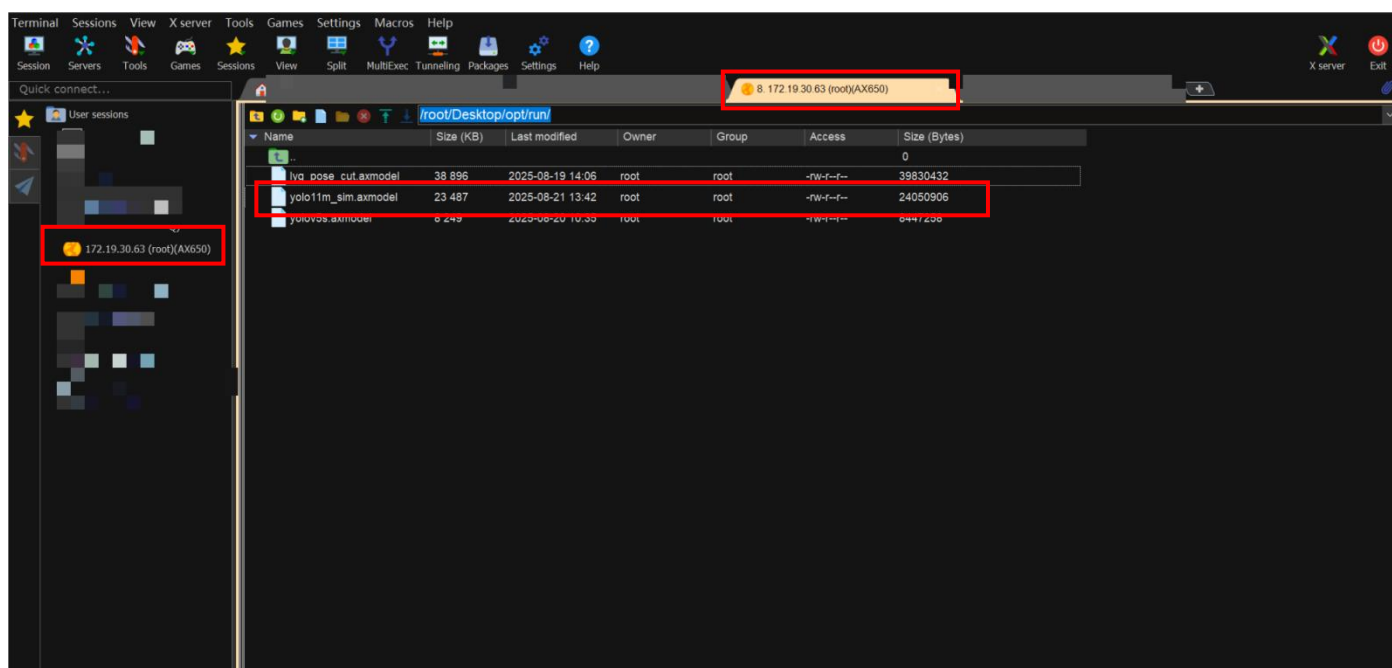
(9) 運行程式

1. 把模型放到 AX650N

(1) 進入 AX650N 的 /root/Desktop 目錄創建放模型的文件夾，創建並進入 /opt/run

```
cd /root/Desktop
mkdir opt/models && cd /opt/models
```

(2) 開啓 SFTP 把模型傳到 models 文件夾中(可以直接把文件拖到 SFTP 介面中)



2. 進入 HDMI 連接的 AX650N 視窗介面

(1) 按滑鼠右鍵 → Open Terminal Here

(2) 進入程式文件夾

```
cd root/lyg-sample/build/install/ax650
```

(3) 執行運行指令

```
./yolo11detect -m /root/Desktop/opt/models/yolo11m_sim.axmodel -v rtsp://172.xxx.xxx.xxx:8554/lygstream
```

常見問題:

- ◆ 因為程式是使用完整的 opencv 編譯，所以在 SSH 連線無法執行運行指令。請在 AX650 介面執行運行指令或使用 headless 的 opencv 編譯。
- ◆ 執行運行指令後顯示無法連接 RTSP 相機，請檢查 AX650 和 RTSP 相機 IP 前 3 組數字是一樣。

以下是兩種常見情況:

(1) AX650 的 IP 被修改為固定 IP，檢查 interfaces 檔案:

```
vi /etc/network/interfaces
```

改為 DHCP 獲取 IP

```
allow-hotplug eth0
iface eth0 inet dhcp <-- 加入此代碼

# 註解以下代碼
# iface eth0 inet static
#   address 192.168.xxx.xxx
#   netmask 255.255.255.0
#   gateway 192.168.xxx.1
```

按

```
esc
```

再輸入

```
:wq
reboot
```

重新啟動後輸入

```
Ip addr show eth0
```

將會顯示新的 IP。

(2) RTSP 相機如果被設定為固定 IP:

以本公司使用的 RTSP 相機為例，如果已設定為固定 IP 可用以下方法切換為 DHCP

```
vi /etc/init.d/rcS
ifconfig eth0 hw ether xx:xx:xx:xx:xx:xx <----記下此 Mac Address
udhcpc -i eth0 <----加入此代碼
```

```
vi /etc/init.d/net_config
```

請註解以下代碼:

```
# # eth0
# ipaddr0=192.168.10.13
# netdev0=eth0
# netmask0=255.255.255.0
# gateway0=192.168.10.1
```

按

```
esc
```

再輸入

```
:wq
reboot
```

下載 cs_mac_x64.exe(MAC Address Scanner)

掃描網段應顯示為 172.xxx.xxx.0/255.255.255.0 (xxx 為公司網絡的 IP)。



在 MAC 地址下找到記下的 MAC Address，對應的 IP 地址就是 RTSP 相機的 IP。

IP地址	MAC地址	主机名	工作组	网卡厂商	比较结果
17					新IP地址与MAC地址